DOCUMENTO DE ENTREGABLE DE SOFTWARE

Software Deliverable Document (SDD)

Nombre del Proyecto	**Sistema de ventas**
Versión del Documento	**v1.0**
Fecha de Entrega	**16/04/2026**
Curso / Asignatura	**Programación 5**
Institución	**Universidad Tecnológica Costarricense**
Profesor	**Ing. Pablo Cordero Vega**
Equipo / Grupo	**Grupo 2**

INTEGRANTES DEL EQUIPO

Nombre Completo	Teléfono	Rol en el Proyecto	Correo Electrónico
Michael Ezequiel Ávila Hernández	**61136987**	**Dev**	**mavilah0271@uvirtual.ac.cr**
Danna Sofia Rojas Zumbado	**71487360**	**Dev**	**drojasz0231@uvirtual.ac.cr**
José Steven Alvarado Hernández	**71291445**	**Dev**	**jalvaradoh0593@uvirtual.ac.cr**
Isaac Rodríguez Víquez	**85102283**	**Dev**	**irodriguezv0471@uvirtual.ac.cr**
Carlos De La Torre	**63521985**	**Dev**	**cdelatorrep0890@uvirtual.ac.cr**

0. Historial de Versiones

Versión	Fecha	Autor(es)	Descripción del Cambio
v0.1	**09/04/2026**	**Isaac Rodríguez Víquez**	**Creación inicial del documento**
v1.0	**16/04/2026**	**Isaac Rodríguez Víquez, Danna Sofia Rojas Zumbado, Michael Ezequiel Ávila Hernández**	**Versión final para entrega**

1. Introducción

1.1 Propósito del Documento

Este documento describe el entregable de software desarrollado para el curso Programación 5. Su objetivo es presentar de manera formal los artefactos, decisiones de diseño, arquitectura y resultados del proyecto de software.

El objetivo principal del sistema es brindar servicios de gestión, ventas y reportes a los usuarios, permitiendo llevar un control automatizado en sus procesos.

El sistema se ha desarrollado para ser utilizado por usuarios como administradores, cajeros y auditores. Cada tipo de usuario tiene permisos diferentes: el administrador puede acceder a todas las funciones, el cajero solo a las ventas y consultas básicas, y el auditor puede ver reportes sin modificar datos. Esto permite que la información esté más segura y que cada quien haga solo lo que le corresponde.

Entre las principales funciones del sistema se encuentran:

- · Gestión de productos y clientes (agregar, consultar, actualizar y eliminar).
- · Registro de ventas, donde el sistema calcula automáticamente el total y descuenta el stock de los productos vendidos.
- · Generación de reportes, que ayudan a ver las ventas, productos más vendidos o movimientos importantes para tomar decisiones.

Para su desarrollo se utilizó el lenguaje C# en Visual Studio, con WinForms para la parte visual (pantallas y botones) y MySQL como base de datos para guardar toda la información. Además, se aplicó una arquitectura en capas (presentación, lógica de negocio y acceso a datos), lo que hace que el código sea más ordenado, fácil de mantener y de crecer en el futuro.

1.2 Alcance del Proyecto

El sistema de ventas tiene como propósito gestionar de manera eficiente las operaciones comerciales de una organización, permitiendo el control de productos, clientes y ventas, así como la generación de reportes y la administración de usuarios según su rol. El software abarca las siguientes funcionalidades principales:

- Gestión de productos (registro, consulta, actualización y eliminación).
- Gestión de clientes (registro, consulta, actualización y eliminación).
- Registro de ventas con cálculo automático de totales y actualización de stock
- Generación de reportes de ventas, clientes y productos con opción de exportación.
- Control de acceso mediante roles de usuario (administrador, cajero y auditor).

1.3 Definiciones, Siglas y Abreviaturas

Término / Sigla	Definición
CRUD	Create, Read, Update, Delete – operaciones básicas de gestión de datos
UI	Interfaz de Usuario – parte del sistema con la que interactúa el usuario (formularios, botones, etc.).
POO	Paradigma de programación basado en el uso de clases y objetos para organizar el código.
LINQ	Herramienta de C# que permite realizar consultas a colecciones de datos de forma sencilla.
JSON	Formato de texto utilizado para almacenar y transportar datos de manera estructurada.
MySQL	Sistema de gestión de bases de datos relacional utilizado para almacenar la información del sistema.
WinForms	Tecnología de .NET utilizada para crear interfaces gráficas de escritorio.
DAO	Patrón de diseño que se encarga de acceder a la base de datos y manejar las operaciones de datos.
DTO	Objeto utilizado para transportar datos entre diferentes capas del sistema.
Repositorio	Patrón que centraliza el acceso a los datos, facilitando su manejo y mantenimiento.

Factory	Patrón de diseño que permite crear objetos sin especificar la clase exacta.
Arquitectura en Capas	Modelo que divide el sistema en partes (Presentación, Negocio y Datos) para organizar mejor el código.
Capa de Presentación	Parte del sistema que interactúa directamente con el usuario (interfaz gráfica).
Capa de Negocio	Contiene la lógica del sistema, como cálculos y reglas de negocio.
Capa de Datos	Se encarga de la comunicación con la base de datos.
Git	Sistema de control de versiones que permite registrar cambios en el código.
GitHub	Plataforma en línea donde se almacenan repositorios de código usando Git.
DataGridView	Componente de WinForms que permite mostrar datos en forma de tabla.
SQL Injection	Tipo de ataque que intenta manipular consultas a la base de datos; se previene usando parámetros.

1.4 Visión General del Documento

El documento está organizado de la siguiente manera: la Sección 2 describe los requisitos del sistema; la Sección 3 presenta la arquitectura y diseño; la Sección 4 muestra los diagramas UML; la Sección 5 describe la implementación; la Sección 6 explica las pruebas realizadas; y la Sección 7 presenta las conclusiones.

2. Requisitos del Sistema

El sistema de ventas propuesto busca facilitar la gestión de productos, clientes y transacciones de manera ordenada y eficiente. Permitirá a los usuarios registrar información, realizar ventas y consultar datos importantes sin necesidad de procesos complicados. Además, el sistema almacenará la información de forma segura en una base de datos, generará reportes útiles y contará con una interfaz amigable que facilite su uso incluso para personas con conocimientos básicos en computación. Todo esto con el objetivo de simular un entorno comercial real de manera práctica.

2.1 Requisitos Funcionales

Los requisitos funcionales describen lo que el sistema debe hacer.

ID	Nombre	Descripción	Prioridad
RF-01	Gestión de productos	El sistema debe permitir al usuario registrar, consultar, actualizar y eliminar productos. Cada producto debe contener código único, nombre, descripción, precio y cantidad en stock. Además, el sistema debe validar que no existan códigos duplicados y que los valores ingresados sean correctos.	Alta
RF-02	Gestión de clientes	El sistema debe permitir al usuario	Alta

		registrar, consultar, actualizar y eliminar clientes. Cada cliente debe incluir ID único, nombre completo, número de teléfono y correo electrónico. El sistema debe validar el formato del correo electrónico.	
RF-03	Registro de ventas	El sistema debe permitir registrar ventas mediante la selección de un cliente y uno o varios productos. Debe registrar fecha, cantidades y calcular automáticamente el total de la venta. Además, debe actualizar el stock de los productos vendidos y validar disponibilidad antes de confirmar la venta.	Alta
RF-04	Cálculo automático	El sistema debe calcular automáticamente el total de la venta	Alta

		multiplicando el precio de cada producto por su cantidad, sumando todos los valores y mostrando el total final al usuario, así como el descuento y el IVA.	
RF-05	Consulta de ventas	El usuario debe poder visualizar el historial de ventas realizadas, con información detallada de cada transacción.	Media
RF-06	Generación de reportes	El sistema debe generar reportes con JSON que permitan analizar la información, como: • Ventas realizadas en un período • Productos más vendidos • Lista de clientes	Media

RF-07	Validar datos	El sistema debe verificar que la información ingresada sea correcta, por ejemplo: • No permitir campos vacíos • Validar formatos (correo, números, letras, etc.) • Evitar valores negativos en precios o cantidades	Alta
RF-08	Autenticación	El sistema debe permitir el acceso mediante autenticación de usuarios (inicio de sesión con usuario y contraseña). Debe validar credenciales y restringir el acceso a usuarios no autorizados.	Baja

2.2 Requisitos No Funcionales

Los requisitos no funcionales describen las características de calidad del sistema (rendimiento, seguridad, usabilidad, etc.).

ID	Categoría	Descripción
RNF-01	Usabilidad	El sistema debe ser fácil de usar, con una interfaz clara, botones bien identificados y navegación sencilla.
RNF-02	Rendimiento	Las operaciones deben ejecutarse rápidamente, evitando tiempos de espera prolongados.
RNF-03	Seguridad	La información debe estar protegida, evitando accesos no autorizados y asegurando la integridad de los datos.
RNF-04	Mantenibilidad	El código debe estar bien estructurado y documentado para facilitar futuras modificaciones o mejoras.
RNF-05	Escalabilidad	El sistema debe permitir agregar nuevas funcionalidades sin afectar su funcionamiento.
RNF-06	Disponibilidad	El sistema debe estar disponible cuando el usuario

		lo necesite, sin fallos constantes.
RNF-07	Compatibilidad	Debe funcionar correctamente en el entorno definido (Windows con .NET).
RNF-08	Integridad	Debe garantizar que la información almacenada sea correcta y consistente.
RNF-09	Portabilidad	Debe poder instalarse en diferentes computadoras sin complicaciones.
RNF-10	Documentación	Debe incluir documentación clara para desarrolladores y usuarios.

2.3 Restricciones del Sistema

- Hardware: mínimo 4GB RAM.
- Software: Windows, Visual Studio, .NET 6.
- Base de datos: MySQL.
- Arquitectura en capas obligatoria.
- Uso de Git y GitHub.
- Interfaz en WinForms.
- Cumplimiento de lineamientos del curso.

3. Arquitectura y Diseño del Sistema

3.1 Descripción General de la Arquitectura

El sistema de ventas está desarrollado utilizando una arquitectura en capas, lo que permite organizar el código de forma estructurada, separando responsabilidades y facilitando el mantenimiento, escalabilidad y comprensión del sistema.

Las capas principales del sistema son:

1. Capa de Presentación (UI – Interfaz de Usuario)

Esta capa es la encargada de la interacción directa con el usuario. Está compuesta por formularios desarrollados en Windows Forms (WinForms), donde el usuario puede ingresar, visualizar y modificar información.

Responsabilidades:

- Mostrar información en pantalla (productos, clientes, ventas).
- Capturar datos ingresados por el usuario.
- Validar datos básicos (campos vacíos, formatos).
- Enviar solicitudes a la capa de negocio.

Ejemplo: Formulario de registro de ventas donde el usuario selecciona productos y cantidades.

2. Capa de Negocio (Lógica del Sistema)

Esta capa contiene toda la lógica del sistema y las reglas de negocio. Se encarga de procesar la información recibida desde la capa de presentación antes de enviarla a la base de datos.

Responsabilidades:

- Aplicar reglas del negocio (ej: cálculo de totales).
- Validar datos de forma más completa.
- Coordinar operaciones entre la interfaz y la base de datos.
- Utilizar patrones como DTO, Factory y Repository.

Ejemplo:

Calcular el total de una venta sumando los productos seleccionados.

3. Capa de Datos (Acceso a Base de Datos)

Esta capa se encarga de la comunicación directa con la base de datos MySQL. Aquí se implementan los patrones DAO y Repository para manejar las operaciones CRUD.

Responsabilidades:

- Insertar, actualizar, eliminar y consultar datos.
- Ejecutar consultas SQL.
- Manejar la conexión a la base de datos.
- Garantizar la integridad de la información.

Ejemplo: Guardar una venta en la base de datos o consultar la lista de productos.

Comunicación entre Capas

- El sistema sigue un flujo de comunicación estructurado:
- El usuario interactúa con la Capa de Presentación (por ejemplo, al registrar una venta).
- La información ingresada se envía a la Capa de Negocio, donde se valida y se procesa.
- La Capa de Negocio utiliza objetos (DTO) y llama a la Capa de Datos para guardar o consultar información.
- La Capa de Datos ejecuta las operaciones en MySQL y devuelve los resultados.
- La información regresa a la Capa de Negocio y luego a la Capa de Presentación, donde se muestra al usuario.

3.2 Diagrama de Arquitectura

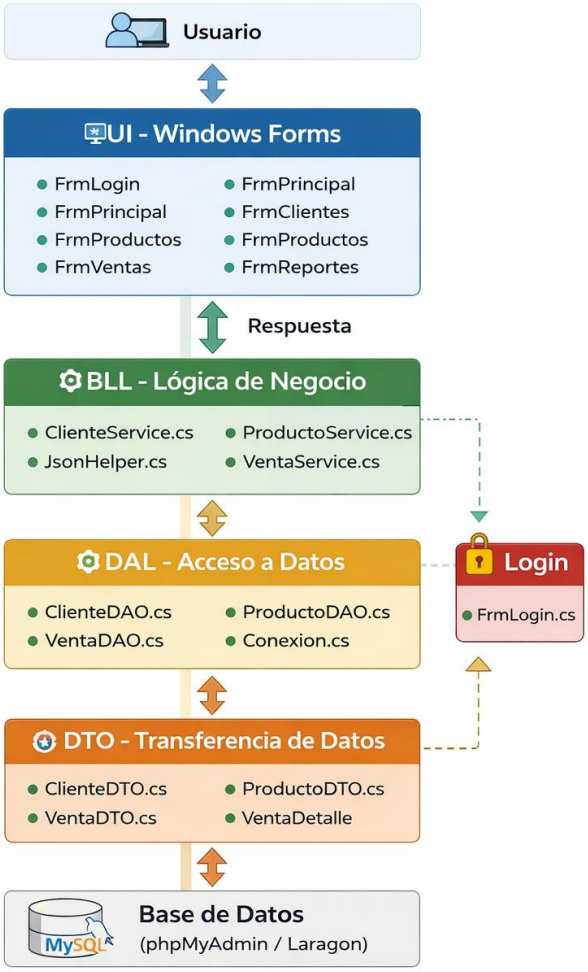

Figura 1: Diagrama de Arquitectura del Sistema

3.3 Tecnologías y Herramientas Utilizadas

Categoría	Tecnología / Herramienta	Versión	Propósito
Lenguaje de programación	C#	10/11	Lógica del sistema
Framework	.NET 6	6.0	Base del sistema
Base de datos	MySQL	8.0	Almacenamiento de datos
Control de versiones	Git / GitHub	N/A	Gestión del código fuente
IDE	Visual Studio	2022	Desarrollo
Herramienta de diseño	StarUML	5.1.0	Diagramas

3.4 Modelo de Base de Datos

El sistema utiliza una base de datos relacional en MySQL para gestionar clientes, productos, ventas y usuarios. Las entidades principales son clientes, productos, ventas y venta_detalle, donde esta última permite resolver la relación entre productos y ventas. También se incluyen usuarios y roles para control de acceso.

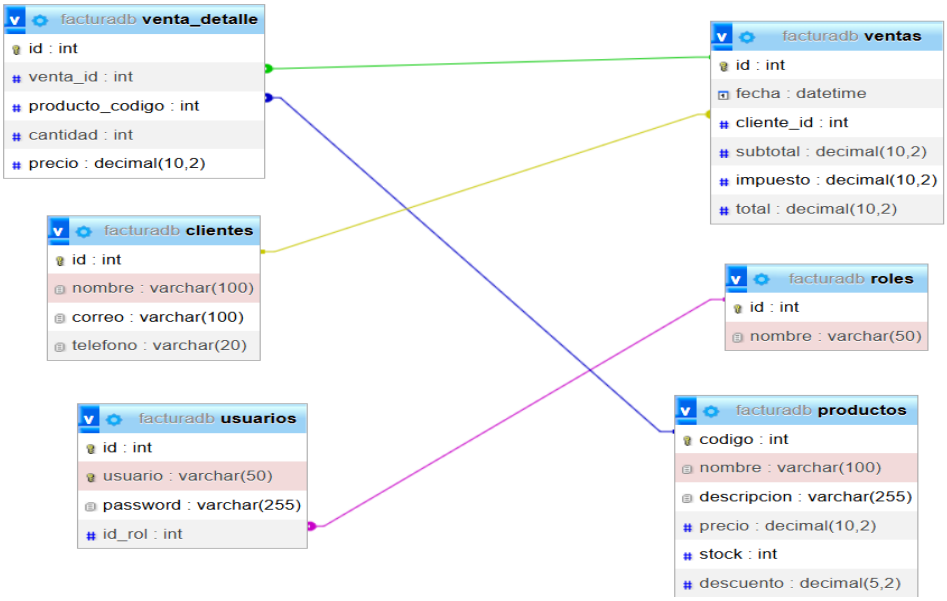

Figura 2: Modelo de Base de Datos

4. Diagrama UML y de capas

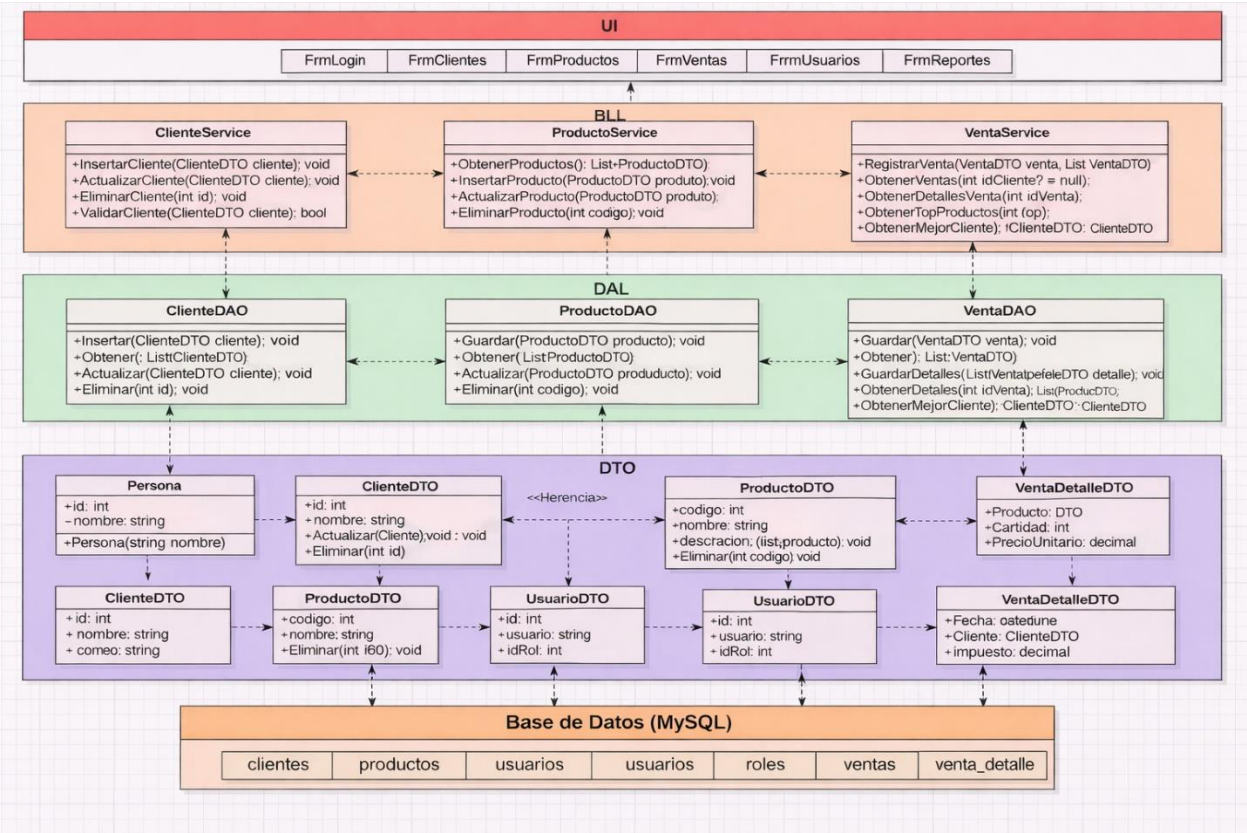

Figura 3: Diagrama UML

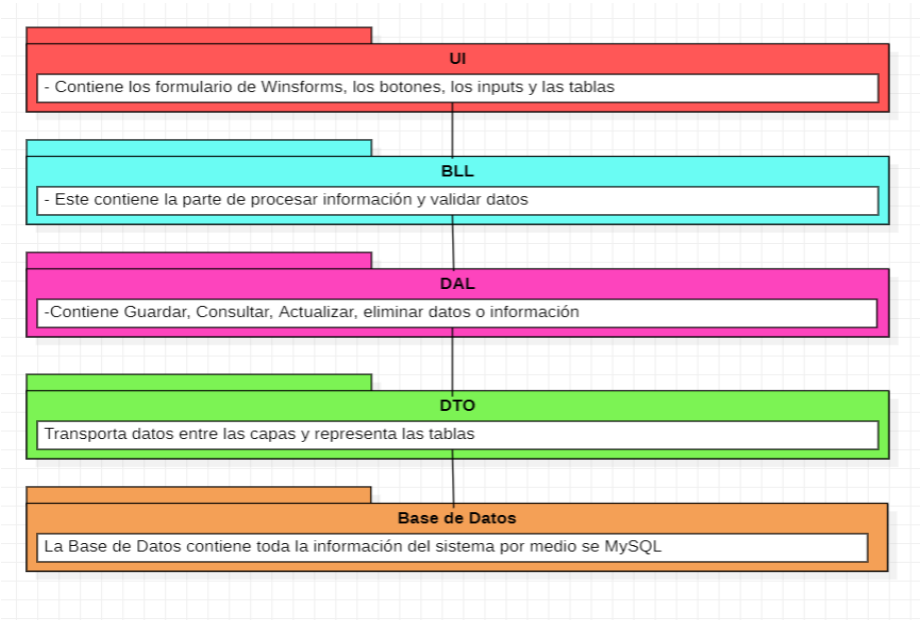

Figura 4: Diagrama de capas

5. Implementación

5.1 Estructura del Proyecto

El proyecto **Sistema Completo de Ventas** está desarrollado en C# bajo la plataforma .NET, utilizando una **arquitectura en capas** que separa claramente las responsabilidades del sistema. La solución está compuesta por cinco proyectos independientes que se comunican entre sí de forma unidireccional.

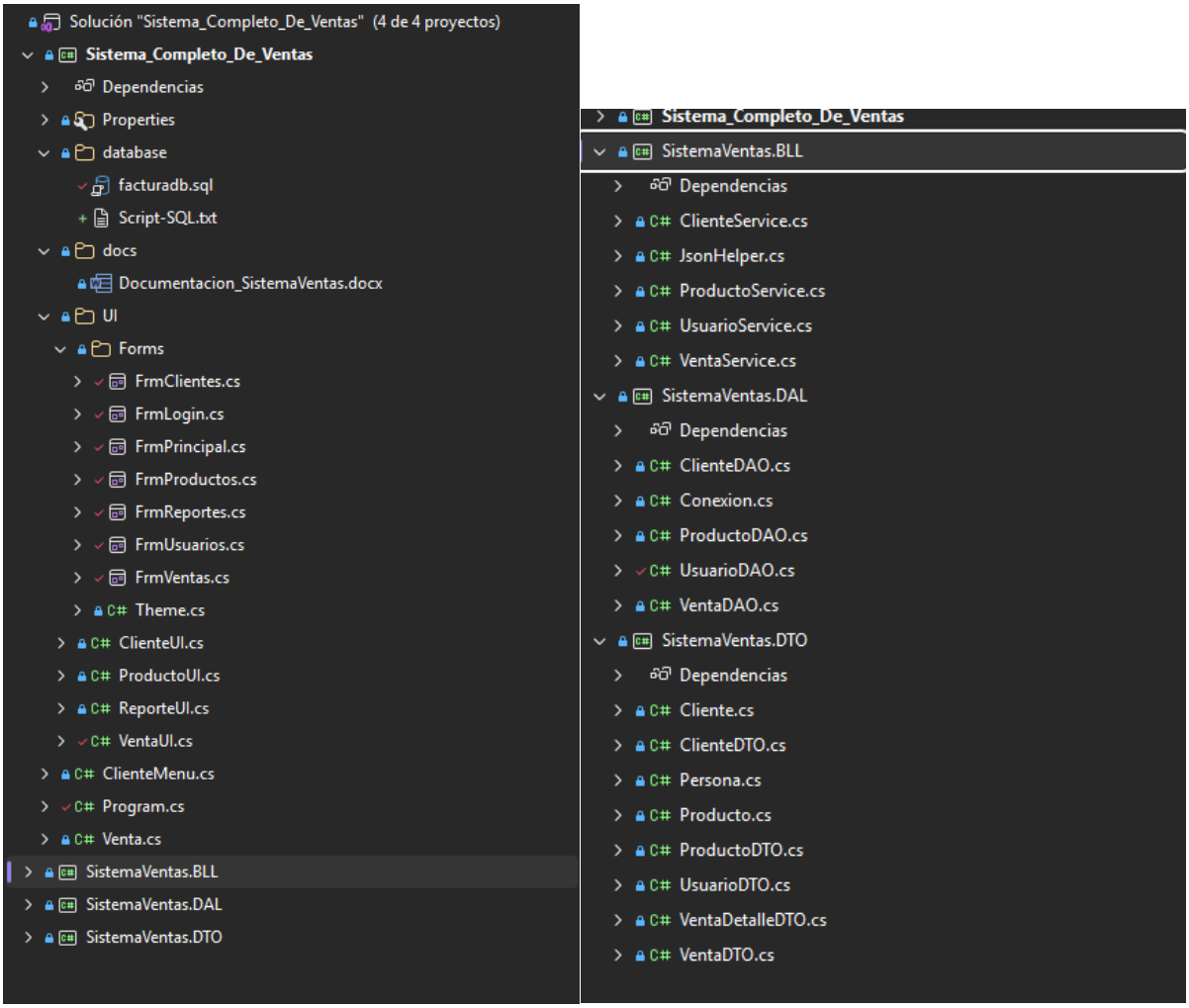

Figura 5 y 6: Estructura del proyecto

5.2 Patrones de Diseño Utilizados

Patrón	Descripción	Dónde se aplicó
Arquitectura en Capas	Divide el sistema en Presentación, Negocio y Datos para organizar mejor el código y separar responsabilidades.	En toda la aplicación
DAO	Permite separar la lógica de acceso a datos del resto del sistema. Se encarga de realizar operaciones con la base de datos.	Clases de acceso a datos (ProductoDAO, ClienteDAO, etc.)
DTO	Se utiliza para transportar datos entre capas sin exponer directamente las entidades de la base de datos.	Objetos como ProductoDTO, ClienteDTO, VentaDTO
Repositorio	Centraliza el acceso a los datos y facilita la manipulación de la información, desacoplando la lógica de negocio.	Capa de datos (repositorios de productos, clientes, ventas)
Factory	Permite crear objetos sin especificar la clase exacta, facilitando la flexibilidad del sistema.	Creación de objetos en la capa de negocio
Validaciones	Asegura que los datos ingresados cumplan con reglas antes de ser procesados.	Formularios de productos, clientes y ventas
Manejo de Excepciones		En toda la aplicación (try-catch en operaciones)

	Permite controlar errores durante la ejecución del sistema para evitar fallos inesperados.	

5.3 Descripción de Módulos o Componentes Principales

Módulo / Clase	Responsabilidad	Tecnología
Productos	Se encarga de gestionar toda la información relacionada con los productos, como registrar nuevos productos, editar sus datos, eliminarlos y consultarlos. También controla el stock disponible.	C#, .NET 6, WinForms, MySQL
Clientes	Permite administrar los datos de los clientes, incluyendo su registro, actualización, eliminación y consulta. Facilita el control de quién realiza las compras.	C#, .NET 6, WinForms, MySQL
Ventas	Gestiona el proceso de ventas, permitiendo registrar transacciones, asociar clientes, seleccionar productos, calcular totales y guardar la información.	C#, .NET 6, WinForms, MySQL

Detalle de Ventas	Maneja la relación entre ventas y productos, registrando los productos vendidos en cada transacción, junto con cantidades y precios.	C#, .NET 6, MySQL
Reportes	Genera reportes de ventas, productos y clientes, permitiendo visualizar información importante	C#, .NET 6
Base de Datos	Se encarga de la conexión con MySQL, ejecución de consultas, almacenamiento y recuperación de datos del sistema.	MySQL, C#, ADO.NET
Interfaz de Usuario	Proporciona la interacción con el usuario mediante formularios, botones y tablas, permitiendo el uso del sistema de forma visual e intuitiva.	WinForms (.NET 6)
Autenticación	Controla el acceso al sistema mediante inicio de sesión, validando usuarios y restringiendo el acceso a funciones.	C#, MySQL
Validaciones	Verifica que los datos ingresados sean correctos antes de ser procesados,	C# (.NET 6)

	evitando errores y datos inválidos.	
Lógica de Negocio	Contiene las reglas del sistema, como cálculos de ventas, validaciones y procesos internos.	C# (.NET 6)

5.4 Instrucciones de Instalación y Ejecución

A continuación, se detallan los pasos necesarios para configurar y ejecutar el proyecto:

Requisitos previos

- .NET 6 o superior
- Visual Studio 2022
- MySQL 8
- Git

Clonar el repositorio

- git clone [https://github.com/viquezisaac373-del/-Proyecto_Programacion_5-UTC-2026-G2-Sistema-Ventas.git]

Configuración

- Crear la base de datos en MySQL (facturadb).
- Configurar la cadena de conexión en el proyecto con las credenciales.

Ejecutar el proyecto

- Desde Visual Studio: F5 o seleccionando a la opción que se llama compilar

6. Pruebas del Sistema

6.1 Estrategia de Pruebas

Las pruebas del sistema se realizaron utilizando un enfoque mixto, combinando pruebas manuales y pruebas funcionales. Se optó por pruebas manuales debido a que el sistema fue desarrollado en un entorno WinForms y no se implementaron herramientas de automatización.

Se realizaron pruebas unitarias a nivel lógico verificando el correcto funcionamiento de los métodos principales, así como pruebas de integración para validar la comunicación entre la capa de presentación, la lógica de negocio y la base de datos MySQL.

Además, se aplicaron pruebas de aceptación simulando el comportamiento de los diferentes tipos de usuario (administrador, cajero y auditor), con el fin de verificar que los permisos y funcionalidades asignadas funcionen correctamente.

6.2 Casos de Prueba

ID	Caso de Prueba	Precondición	Pasos	Resultado Esperado	Resultado Obtenido	Estado
CP-01	Inicio de sesión correcto (Administrador)	Usuario registrado en el sistema	- Ingresar usuario y contraseña correctos - Presionar botón de acceder	Acceso al sistema con permisos de administrador	Acceso correcto al sistema con rol administrador	Pasó
CP-02	Inicio de sesión con contraseña incorrecta	Usuario registrado	- Ingresar usuario válido - Ingresar contraseña incorrecta - Intentar	Mostrar mensaje de error y denegar acceso	Muestra mensaje de contraseña incorrecta	Pasó

			iniciar sesión			
CP-03	Restricción de roles (Cajero)	Usuario tipo cajero	- Iniciar sesión como cajero	No visualizar opciones de administración	El sistema restringe correctamente el acceso	Pasó
CP-04	Registro de producto válido	Acceso al módulo de productos	- Ingresar datos válidos - Guardar producto	Producto registrado correctamente	Producto creado y guardado en la base de datos	Pasó
CP-05	Registro de producto con datos incompletos	Acceso al módulo de productos	- Dejar campos vacíos - Intentar guardar	El sistema debe validar e impedir el registro	Permite registrar producto con stock en 0	Falló
CP-06	Edición de producto	Producto existente	- Seleccionar producto - Modificar datos - Guardar cambios	Datos actualizados correctamente	Edición realizada correctamente	Pasó
CP-07	Eliminación de producto	Producto existente	- Seleccionar producto - Eliminar	Producto eliminado del sistema	Eliminación correcta desde la interfaz	Pasó

CP-08	Registro de cliente inválido	Acceso al módulo de clientes	- Ingresar datos inválidos - Guardar	Validación de datos incorrectos	Solicita mínimo de caracteres y formato válido	Pasó
CP-09	Registro de venta	Productos con stock disponible	- Seleccionar cliente - Agregar productos - Confirmar venta	Venta registrada y stock actualizado	Venta registrada correctamente y stock descontado	Pasó
CP-10	Venta con stock insuficiente	Producto con bajo stock	- Intentar vender más de lo disponible	Mostrar error de stock	Mensaje de “stock insuficiente”	Pasó
CP-11	Venta sin productos	Acceso al módulo de ventas	- Intentar registrar venta sin productos	Impedir la operación	Solicita agregar productos y cliente	Pasó
CP-12	Generación de reportes con datos	Existencia de datos	- Generar reporte	Reporte generado correctamente	Reporte exportado en JSON correctamente	Pasó
CP-13	Generación de reportes sin datos	Base de datos sin registros	- Intentar generar reporte	Mostrar advertencia	Mensaje indicando no hay datos para exportar	Pasó

6.3 Cobertura de Pruebas

Las pruebas realizadas fueron de tipo manual, cubriendo aproximadamente el 100% de los requisitos funcionales del sistema, incluyendo los módulos de autenticación, productos, clientes, ventas y reportes. Se verificaron tanto escenarios positivos como negativos para asegurar el correcto funcionamiento del sistema.

6.4 Defectos Encontrados y Correcciones

ID Defecto	Descripción del Bug	Severidad	Estado
BUG-01	El sistema permite registrar productos con datos incompletos, específicamente permitiendo stock en 0 sin validar otros campos obligatorios	Media	Pendiente
BUG-02	No se permite exportar reportes cuando no hay datos, aunque el sistema muestra advertencia (comportamiento controlado pero mejorable)	Baja	Corregido

7. Manual de Usuario

7.1 Descripción General de la Interfaz

Este Sistema cuenta con una interfaz gráfica que se fue desarrollando por medio de Windows Forms, en lo cual se puede ejecutar clientes, productos y ventas de una manera muy sencilla.

En esta interfaz está construido por diferentes módulos que se ejecutan a través de botones o incluyendo menú para que el usuario se le facilite al momento de realizar el sistema.

7.2 Guía de Uso

Figura 7: Login del sistema

Inicio de sesión

El usuario autorizado accede al sistema mediante un nombre de usuario y contraseña, garantizando la seguridad de la información.

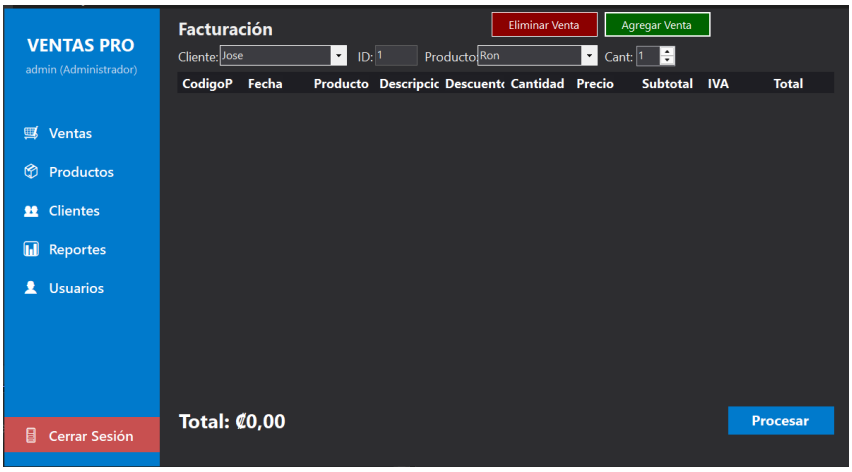

Figura 8: Módulo de ventas

Inicio del proceso de venta

El vendedor selecciona la opción de realizar una nueva venta dentro del sistema.

Figura 9: Módulo de gestión de productos

Gestión de productos

Se registran productos para la venta, incluyendo información como nombre, precio, descripción, cantidad en stock y el descuento.

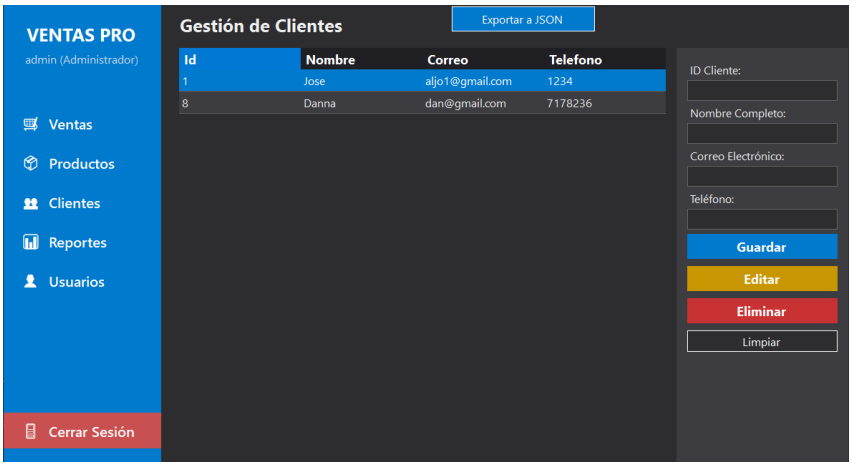

Figura 10: Módulo de gestión de clientes

Gestión de clientes

Primero, el sistema permite registrar los datos del cliente (nombre, cédula, teléfono, correo). Esto sirve para tener control de quién realiza la compra.

Figura 11: Módulo de reportes y estadísticas

Reportes y estadísticas

El sistema permite elegir el tipo de reporte que se desea generar, por ejemplo:

- Cierre de ventas (IVA)
- Ranking de productos
- Auditoria de clientes
- Análisis de utilidad

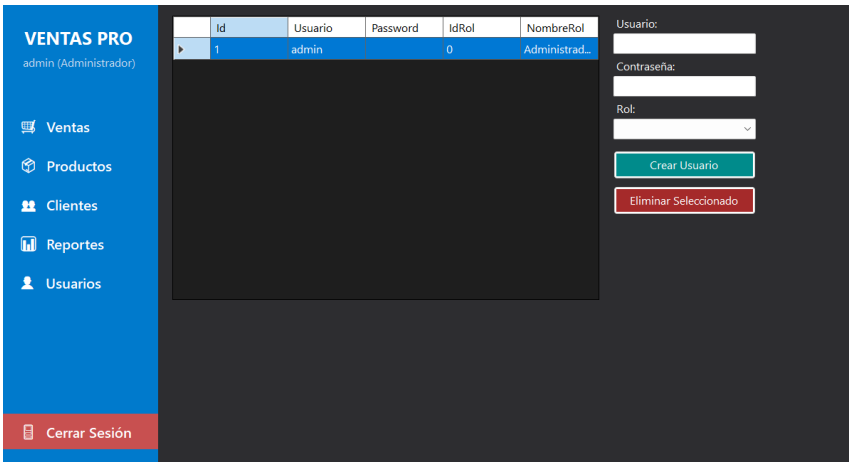

Figura 12: Módulo de registro de nuevos usuarios

Registro de nuevos usuarios

El sistema permite ingresar nuevos usuarios, solicitando datos como:

- Nombre de usuario
- Contraseña
- Rol (administrador, auditor o cajero)

8. Conclusiones y Lecciones Aprendidas

8.1 Conclusiones

Conclusión General

El desarrollo del sistema de ventas permitió cumplir de manera satisfactoria con el objetivo principal del proyecto, logrando implementar una aplicación funcional capaz de gestionar productos, clientes y ventas de forma eficiente. Durante su elaboración, se aplicaron correctamente los conocimientos adquiridos en el curso, tales como la programación orientada a objetos, el manejo de bases de datos con MySQL, la arquitectura en capas y el desarrollo de interfaces gráficas con WinForms, lo que permitió construir un sistema organizado y fácil de utilizar.

Las tecnologías empleadas, como C#, .NET 6 y MySQL, resultaron adecuadas para el desarrollo del sistema, ya que facilitaron la creación de una aplicación estructurada, segura y escalable. Asimismo, el uso de patrones de diseño como DAO, DTO y Repository contribuyó a mejorar la organización del código y su mantenibilidad.

Por otro lado, el trabajo en equipo y el seguimiento de un proceso estructurado fueron fundamentales para el éxito del proyecto. La distribución de tareas, el uso de herramientas como Git y GitHub, y el cumplimiento de los entregables semanales permitieron avanzar de manera ordenada y eficiente. En conjunto, todos estos elementos hicieron posible el desarrollo de un sistema completo, funcional y alineado con los requerimientos establecidos, demostrando la correcta aplicación práctica de los conocimientos adquiridos.

8.2 Lecciones Aprendidas

Reflexión sobre el Aprendizaje y Mejoras

Durante el desarrollo del proyecto se adquirieron importantes aprendizajes tanto a nivel técnico como metodológico. En el aspecto técnico, se logró aplicar de manera práctica conceptos como la programación orientada a objetos, el uso de bases de datos con MySQL, la implementación de arquitectura en capas y el desarrollo de interfaces gráficas con WinForms. Además, se fortalecieron habilidades en el manejo de colecciones, consultas con LINQ, validación de datos, manejo de errores y uso de herramientas de control de versiones como Git y GitHub.

En el ámbito metodológico, se aprendió la importancia de trabajar de forma organizada, siguiendo un proceso estructurado mediante entregables semanales, lo que permitió avanzar progresivamente en el desarrollo del sistema. También se reforzó el trabajo en equipo, la comunicación y la distribución de tareas, aspectos clave para cumplir con los objetivos del proyecto.

Si se volviera a desarrollar el sistema, se podrían implementar algunas mejoras, como una mejor planificación inicial del diseño y la base de datos, una distribución más equilibrada de las tareas desde el inicio, y una mayor implementación de pruebas unitarias para asegurar la calidad del sistema. Asimismo, se podría mejorar la interfaz de usuario y agregar funcionalidades adicionales, como un sistema de autenticación más completo o reportes más avanzados.

8.3 Trabajo Futuro

Para una próxima versión del sistema de ventas, se podrían implementar diversas mejoras que permitan ampliar sus capacidades y hacerlo más completo y profesional.

Mejora o funcionalidad futura: Reportes avanzados y dashboard interactivo: Se podrían desarrollar reportes más detallados y visuales, incluyendo gráficos dinámicos (ventas por mes, productos más vendidos, ingresos, etc.), permitiendo una mejor toma de decisiones basada en datos.